LOANLENS AI

Loan Readiness and EMI Planning Platform

AI SPECIALIST - BFSI CAPSTONE PROJECT

Submitted by	Pushpraj Patil
Project domain	Banking, Financial Services and Insurance
Repository	github.com/patilpushpraj39-lang/loanlens-ai
Live demo	loanlens-ai-henna.vercel.app
Submission date	26 October 2026

Educational capstone demonstration. Results are estimates and are not lending decisions or financial advice.

1. Executive Summary

LoanLens AI is a responsive web application for exploring the cost and affordability of a proposed loan. It combines a standard EMI calculation with debt-load analysis, user-provided credit-score context, repayment breakdowns, practical guidance, scenario comparison and browser-local plan storage.

The project is designed for education and comparison. It does not claim to fetch an official credit score, approve a loan or reproduce any bank's underwriting policy. Each readiness result is accompanied by visible reasons so the user can understand how the entered values affected the outcome.

Problem statement

An EMI figure alone is not enough for a borrowing decision. A user also needs to understand total interest, total repayment, the effect of existing debt and the trade-off between a lower monthly payment and a longer repayment period.

Objectives

- Calculate EMI, total interest and total repayment from user inputs.
- Show monthly debt load using income, existing EMIs and the proposed loan EMI.
- Explain the result using transparent and editable demo rules.
- Allow side-by-side scenario comparison through the What-if Lab.
- Keep the public demo safe by avoiding sensitive personal and banking data.

6	100%	0
USER-FACING MODULES	RESPONSIVE WEB UI	API KEYS NEEDED FOR CORE DEMO

2. Functional Modules

Module	Inputs	Main output
Loan Planner	Amount, rate, tenure, income, existing EMIs, age, employment, experience and optional score	EMI, debt load, total interest, repayment and explained readiness result
Credit Context	Score entered from the user's own report	Educational score band and limitations
EMI Breakdown	Current loan plan	Year-by-year principal, interest and remaining balance
Financial Tips	Calculated plan	Rule-based suggestions linked to the scenario
What-if Lab	Adjusted amount, tenure and existing debt	Pinned baseline compared with the live plan
Saved Plans	User-selected plan label and figures	Browser-local save, reload and deletion

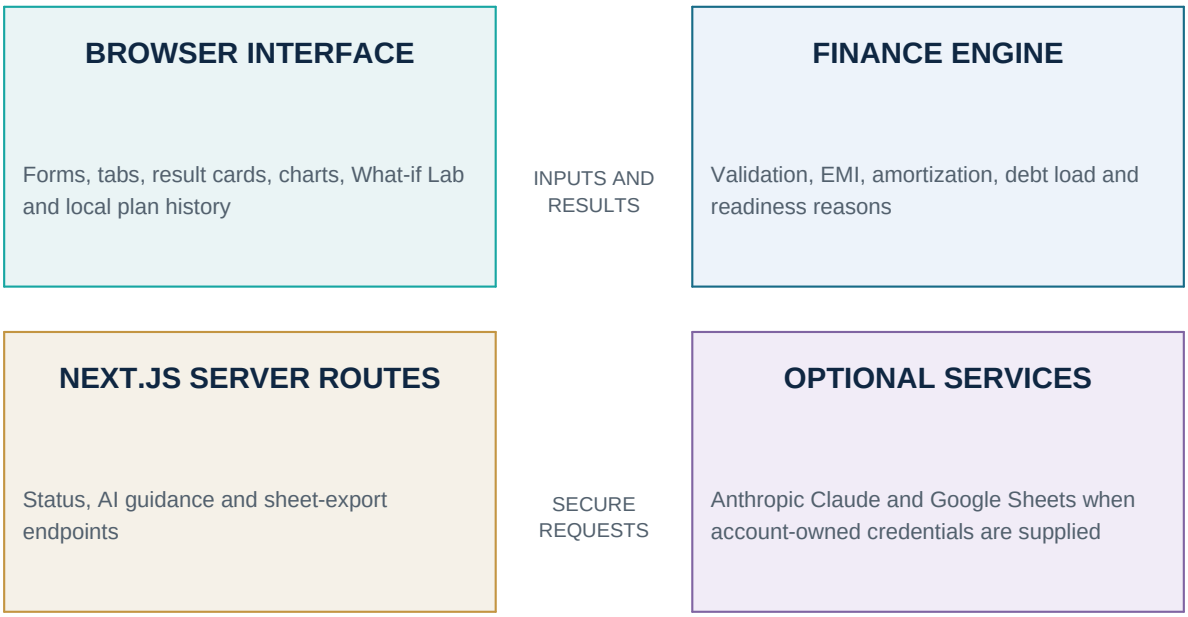
Typical user flow

1	Enter or edit a loan plan
2	Validate the financial inputs
3	Calculate EMI, repayment and debt load
4	Apply transparent educational rules
5	Display reasons, tips and amortization
6	Compare changes or save a fictional plan

The interface updates the result as the user changes values, reducing repeated form submissions and making the effect of each input easier to observe.

3. System Design and Architecture

The application separates the interface, deterministic financial logic and optional external services. The core calculation remains available even when no third-party credentials are configured.



Technology stack

Layer	Technology	Purpose
Frontend	Next.js 16, React 19, TypeScript	Responsive application and typed components
Styling	Tailwind CSS	Consistent responsive layout and visual system
Visualization	Recharts	Repayment and comparison visualizations
Logic	TypeScript finance utilities	Deterministic calculations and validation
Hosting	Vercel	Public production deployment
Optional services	Claude API and Google Sheets API	Explanations and user-selected export

4. Financial Logic

EMI calculation

For principal P, monthly interest rate r and tenure n months:

$$EMI = P \times r \times (1 + r)^n / ((1 + r)^n - 1)$$

The monthly rate is the annual percentage rate divided by 1200. For a zero-interest loan, EMI is P divided by n. Total repayment is EMI multiplied by n, and total interest is total repayment minus principal.

Debt-load calculation

Debt load = (existing monthly EMIs + proposed EMI) / monthly income

Debt load helps the user see how much of the entered monthly income would be committed to debt payments. The displayed thresholds are illustrative markers for the capstone demonstration. They are not lender-specific approval criteria.

Sample verification

Input	Value
Principal	INR 5,00,000
Annual interest rate	12%
Tenure	36 months
Calculated monthly EMI	Approximately INR 16,607.15

Readiness explanation

- Age is checked against the illustrative supported range at loan maturity.
- Work history and employment are included as visible context.
- A credit score is considered only when the user enters one.
- Every negative or review result is accompanied by a reason.

No probability of approval is generated. This avoids presenting a simple rule set as a trained underwriting model.

5. Implementation Details

The source repository follows a standard Next.js layout. Financial rules are kept outside the interface components so the calculations can be reviewed independently.

Path	Responsibility
app/page.tsx	Main workspace, module navigation and user interactions
app/api/insights/route.ts	Optional AI explanation route with an unconfigured safe state
app/api/sheet-sync/route.ts	Optional user-selected Google Sheets export route
app/api/status/route.ts	Integration availability status
components/	Reusable interface components
lib/finance.ts	Validation, EMI, amortization and readiness calculations
lib/sheets.ts	Google Sheets helper logic
docs/	Capstone report and delivery checklist

Input handling

- Number fields are bounded and validated before financial calculations are shown.
- Invalid or incomplete values produce controlled interface states.
- Sample values are fictional and can be replaced during the demonstration.

Plan storage

Saved plans use browser local storage. They remain on the same browser and device until the user deletes them or clears site data. The public demo does not claim cross-device synchronization.

External integrations

Claude and Google Sheets routes are implemented as optional server-side integrations. The public demo intentionally leaves paid credentials unconfigured. The core planner continues to work without them, and secrets are never included in the repository.

6. Testing and Verification

Test area	Verification	Status
Public deployment	Vercel page loads with the planner and sample results	Passed
Standard EMI	Formula checked using INR 5,00,000, 12%, 36 months	Passed
Zero-interest loan	Uses principal divided by tenure	Passed
Amortization	Principal and interest reduce the remaining balance	Passed
Debt-load response	Result changes when income or existing EMI changes	Passed
Invalid input	Boundary and missing-value states are handled	Passed
TypeScript	Project type check completed	Passed
Production build	Vercel-targeted build completed	Passed
Claude response	Requires account-owned API credentials	Optional
Google Sheets export	Requires service-account and sheet credentials	Optional

Current verification scope

The calculations, TypeScript checks, production build and public Vercel page have been verified. The optional paid integrations are not claimed as live because their credentials are not stored in the public project.

Known limitations

- The readiness rules are educational assumptions rather than a trained credit-risk model.
- Credit information is entered by the user and is not verified against a bureau.
- Saved plans are local to one browser unless an optional export is configured.
- The application does not include user accounts or a lender application workflow.

7. Privacy, Security and Responsible Use

The project is intentionally limited to an educational planning use case. Its public interface does not request the user's name, phone number, bank-account number, identity document or login credentials.

Controls used

- Third-party secrets are read only from server-side environment variables.
- The repository includes variable names and setup guidance, not secret values.
- The calculator continues safely when integrations are unavailable.
- The interface labels readiness results as illustrative and explains their causes.
- The credit context clearly states that the entered score is not fetched or verified.

Before enabling paid or persistent services

- Add authentication and authorization to external-service endpoints.
- Apply rate limits and usage monitoring to AI requests.
- Define retention, deletion and consent policies for stored financial scenarios.
- Use a private Google Sheet with least-privilege service-account access.

Responsible-use statement

LoanLens AI must not be used to make or automate a real credit decision. A lender's published terms, verified customer information and applicable regulations are required for an actual loan process.

8. Conclusion and Future Scope

LoanLens AI meets the capstone objective of presenting multiple BFSI decision-support tools in one accessible application. Its main strength is transparency: the user can see the calculation, the debt-load effect, the reason for a result and the consequences of changing the loan plan.

The live application and public source repository make the work directly reviewable. Core functionality does not depend on private services, which keeps the submission reproducible and avoids exposing credentials.

Future enhancements

- Authenticated accounts with cross-device plan history.
- PDF comparison reports generated from selected scenarios.
- Lender-supplied rule profiles with visible versioning.
- Authorized credit-bureau integration with explicit consent.
- Automated unit, accessibility and browser regression tests.
- Audited security controls before any persistent financial-data storage.

Project links

Resource	URL
GitHub repository	https://github.com/patilpushpraj39-lang/loanlens-ai
Live Vercel demo	https://loanlens-ai-henna.vercel.app/

END OF REPORT

Prepared for final capstone evaluation - 26 October 2026